Firma AI — Open Source Release

Component Reference · v1

> This document describes every component included in the Firma AI open-source release (Apache 2.0). It is the primary reference for the team building on top of the OSS package. It explicitly calls out V1 scope boundaries, failure modes, and what is intentionally excluded.

1. Security Boundary — Containment vs Policy Enforcement

Before describing any component, this distinction must be stated clearly:

Firma Sidecar (this OSS package)	Sandbox / host runtime
L7 / application-layer policy enforcement	Process / OS-level containment
Decides what agent calls are allowed, denied, or aborted	Decides what code the agent process can execute
Enforces Cedar policies, capability scope, budget limits	Enforces filesystem, network, syscall restrictions
Acts after the agent has decided to make a call	Acts on the agent runtime itself
Can be replaced by config change (Mini Auth → FA)	Orthogonal to Firma; managed by the deployment platform

> The Sidecar is NOT the hard security boundary for process isolation. It enforces what an agent is authorised to do at the API/call level. The sandbox (container isolation, seccomp, etc.) is the boundary for runtime containment. Both are required; they do not substitute for each other.

Anyone reading this OSS package should not conclude that deploying the Sidecar alone provides a complete security model. It provides the policy enforcement layer. The containment layer is out of scope for this release.

2. Execution Pattern

Every agent call flows through five stages inside the Sidecar, in sequence. The Authority is involved only once per session (pre-flight); it is never contacted during call execution.

Stage 1 — Capability Validation	First enforcement phase. Validates the capability token: parse, signature verify, revocation check. Fully in-process, no network call. Runs in under 1 ms.
Authority (pre-flight only)	Called once at session start to issue a signed capability token. Streams policy bundle and revocation updates in background. Not contacted again on the hot path.
Stage 2 — Constraint / Policy Enforcement	Second enforcement phase (CEE). Builds Cedar context, evaluates Cedar policies, applies budget / scope / threshold checks. Fully local.
Connector	Translates the Execution Envelope to the target protocol. Applies target-specific technical constraints. Normalises the audit event.
External	The downstream system: LLM API, database, third-party service, etc.

> Stage 1 and Stage 2 are two phases of the same Sidecar process, not two separate components. "Gate appears twice" in the architecture diagram is shorthand for these two sequential enforcement phases within a single binary.

3. Mini Authority

Mini Authority is the reference implementation of the Firma Authority interface. Its sole purpose is to make the full OSS stack runnable locally without any cloud service. It is intentionally minimal: no risk engine, no trust graph, no multi-tenancy.

3.1 Cedar policy loader

Reads every .cedar file from /policies at process startup. No hot-reload on the loader itself. Policy updates reach the Sidecar via the WatchPolicyBundle stream. File changes require a process restart to take effect in the loader, but the stream will push the new bundle to connected Sidecars automatically.

3.2 IssueCapability RPC

The core gRPC endpoint consumed by the Sidecar at session start (pre-flight). Receives agent identity, requested actions, resource scope, and session metadata. Evaluates the request against loaded Cedar policies. On success, returns a signed capability token (PASETO v4 or JWT RS256). On denial, returns a reason code.

3.3 WatchPolicyBundle RPC

Persistent server-streaming gRPC call. Immediately streams the current policy bundle on connect, then pushes incremental updates. In Mini Authority this is driven by filesystem watch on /policies. If this stream disconnects and the Sidecar's cached bundle TTL expires (default 30 s), the Sidecar enters fail-closed mode and denies all new requests.

3.4 WatchRevocations

Persistent server-streaming gRPC call that pushes revocation events. In Mini Authority this is mock / file-based: write to revocations.json, Mini Authority streams the update. The Sidecar Stage 1 maintains a bloom filter and LRU cache from this stream, enabling sub-microsecond revocation checks with no network I/O.

3.5 Capability token generation

Generates capability tokens using the Capability Library (see section 5). Two formats: PASETO v4 (default) and JWT RS256. Token payload: session ID, agent ID, action set, resource scope, issued-at, expiry, and an integrity hash of the Cedar context used at issuance.

> Mini Authority is labeled "reference impl · dev / test / demo · not for production". Never deploy it as a real authority endpoint. In production, swap it for Firma Authority via a single config-file change: firma.authority: FA_URL. The Sidecar binary does not change.

4. Firma Sidecar

Firma Sidecar is the primary OSS component. It is the enforcement layer between an agent and the outside world. Every outbound call passes through it. The Sidecar is a single statically-linked binary with no persistent database; all state is in-memory and re-populated from Authority streams on restart.

4.1 Interceptor

Captures outbound agent traffic before it reaches the external system. Three modes today:

- HTTP proxy (port 8080 default) — agent sets HTTP_PROXY=http://localhost:8080.
- gRPC hook — programmatic interceptor registered within the agent process.
- Unix socket — local socket, avoids port binding in containers.
- eBPF (roadmap) — kernel-level capture, no agent-side config required.

Regardless of interception mode, the output is always a structured Execution Envelope passed to the Stage 1 gate.

4.2 Execution Envelope

The central protocol object of the Firma system. The unit of work flowing from interceptor through both enforcement stages, through the Connector, and into the external system.

`intent`	Action type, target resource identifier, action-specific parameters.
`capability`	Signed capability token issued by the Authority at pre-flight. Used by Stage 1 to verify identity and scope.
`metadata`	Session ID, agent ID, timestamp, trace ID, runtime signals (budget consumed, etc.).
`provenance`	Optional / forward-compatible field (V1: schema placeholder). Intended as a hash chain anchoring this envelope to the session's prior calls. Not implemented in V1 runtime. Teams should treat this field as reserved and not build hard dependencies on it yet.

> V1 note on provenance: the field is present in the schema to keep the protocol forward-compatible, but the runtime does not populate or verify the hash chain in V1. It is a placeholder. If your connector or audit logic reads this field, treat it as optional and nullable.

4.3 Stage 1 — Capability Validation

First enforcement phase. Runs synchronously in the request path. Target latency: under 1 ms. Steps:

- Token parse — deserialise the capability token from the Execution Envelope.
- Signature verification — verify cryptographic signature against the Authority public key. Rejects tampered or forged tokens immediately.
- Revocation check — query the revocation bloom filter (O(1) negative check) + LRU cache for confirmed positives. No network I/O.

If Stage 1 passes, the envelope proceeds to Stage 2. If it fails, the call is denied with a structured DENY response. The Authority is never contacted.

4.4 Stage 2 — Constraint / Policy Enforcement (CEE)

Second enforcement phase. The semantic layer where Cedar policies and quantitative constraints are evaluated. Steps:

- Context build — assembles the Cedar request context from envelope fields, local state, and runtime signals.
- Cedar eval — evaluates against the current policy bundle. Deterministic: same context + same bundle = same decision. Runs in microseconds.

- Budget / scope / threshold checks — applies quantitative constraints via pre-computed Cedar context attributes. In V1 these are static or pre-computed values injected before eval: remaining budget, allowed scope, a risk threshold from the Execution Envelope metadata. There is no dynamic risk engine in the OSS release.

> V1 note on risk: Stage 2 applies threshold-based checks on a static or pre-computed risk attribute. It does not compute risk dynamically. "Risk score" in the OSS context means: a numeric value injected by the agent runtime (or defaulting to 0) checked against a configured threshold in Cedar. A full dynamic risk engine is a proprietary Firma Authority capability, not part of V1 OSS.

Possible outcomes: ALLOW (envelope forwarded to Connector), DENY (call blocked, structured response returned), ABORT (mid-flight kill sent to agent and Connector).

4.5 Local State

`Policy bundle cache`	Current serialised Cedar bundle from WatchPolicyBundle. CEE reads this on every call. Includes version number and TTL. If TTL expires without refresh, Sidecar enters fail-closed mode.
`Revocation cache`	Two-layer: bloom filter for O(1) negative checks, LRU cache for confirmed revocations. Populated from WatchRevocations stream. No network I/O at check time.

Neither cache is persisted to disk. On restart, the Sidecar re-fetches both from Authority streams before accepting traffic.

4.6 Audit Emitter

Serialises every Gate decision into a signed ExecutionEvent and forwards to one or more sinks. Each event contains: event ID (UUID v7), session ID, token ID, agent ID, action, resource, decision (ALLOW / DENY / ABORT), deny reason, gate latency (µs), context hash, bundle version, timestamp (ns), ECDSA signature over all preceding fields.

Four output modes: file (append-only), stdout (default for containers, structured JSON lines), gRPC (streaming to downstream audit service), and WAL on disconnect (events buffered on disk if gRPC stream drops, replayed on reconnect — no audit events are lost).

4.7 Credential Injector

Runs after Stage 2 ALLOW, before the envelope reaches the Connector. Fetches credentials for the target system and injects them transparently. The agent never handles credentials directly. Two modes: Vault client (short-lived secrets from HashiCorp Vault or compatible), and basic credential injection (pre-configured API key / bearer token from Sidecar config).

5. Capability Library

Shared cryptographic utility library used by both the Sidecar and Mini Authority. Pure crypto logic, no network dependency, no side effects. Available in Go, Python, and TypeScript.

`Token validation`	End-to-end validation: deserialise, check format, verify signature, check expiry, check scope against requested action/resource. Returns structured result with reason code on failure.
`Parse + verify + sign`	Low-level primitives: parse token to struct, verify signature with public key, sign payload with private key.
`PASETO v4 / JWT RS256`	Two formats: PASETO v4 (default, Ed25519 or XChaCha20-Poly1305) and JWT RS256 for environments with existing JWT infrastructure. Same logical payload schema.
`Expiry + scope checks`	Helpers to check validity window and whether token scope covers a specific requested operation.
`Helper SDK`	Language wrappers for Go, Python, TypeScript. Idiomatic ergonomics over shared primitives. Used by Sidecar (Go) and Mini Authority (Go). Available to agent developers for offline token inspection.

6. Example Agents

Runnable reference applications demonstrating the full Firma execution flow end-to-end. Not production-grade agents; designed to be readable and copy-paste friendly. Both connect to the Sidecar via HTTP proxy (HTTP_PROXY=http://localhost:8080) with no code-level integration required.

6.1 Python agent

Targets OpenAI, Anthropic, and generic HTTP APIs. Exercises three scenarios:

`ALLOW`	Call satisfies all Cedar policies and CEE constraints. Sidecar forwards it, agent receives a normal response.
`DENY`	Call violates a constraint (disallowed endpoint, budget exceeded, etc.). Sidecar blocks it with a structured denial.
`ABORTED`	Call killed mid-flight by an abort signal from the Authority. Agent receives a connection abort.

6.2 TypeScript / Node agent

Same three scenarios using axios and native fetch. Compatible with LangChain.js and Vercel AI SDK. Structurally identical to the Python agent, confirming Firma is language-agnostic at the agent layer. Both agents log Gate decision and latency for each call.

7. Connector · Adapter Layer

Sits between Stage 2 (CEE) and the external system. Responsible for protocol translation, target-specific technical constraints, and audit normalisation.

`Translate Execution Envelope → target protocol`	Converts the Firma-internal Execution Envelope to the target system's native protocol (HTTP, gRPC, DB query, tool call payload, etc.).
`Connector-specific constraints`	Applies technical constraints specific to the target that are not easily expressible generically in Cedar: rate limits (calls/min to a specific API), response schema validation, format normalisation.
`Normalise audit event`	Enriches the envelope with call outcome (status, latency, response size) and passes it to the audit emitter. Applies memory namespace tagging.

> Boundary rule: the Gate (Stage 2 / CEE) decides general policy. The Connector applies target-specific technical constraints that are not conveniently expressible in Cedar. Do not move policy logic into connectors. If a team starts writing business rules inside a connector "because it is easier", that logic belongs in a Cedar policy and the connector should be refactored. A connector that becomes a second policy engine will break the separation of concerns that makes the whole system auditable.

The OSS release includes a generic HTTP connector. Additional connectors (LLM providers, databases, tool APIs) can be contributed as plugins or built privately.

8. Capability Lifecycle

A capability token has a defined lifecycle. Understanding it is required to build agents that handle revocation and abort correctly.

`ISSUED`	Authority has created and signed the token. Returned to the Sidecar, not yet validated by Stage 1. Transient.

ACTIVE	Stage 1 has validated the token. Session is live. Multi-use: the same token can authorise multiple calls within the session; Stage 2 re-evaluates on every call.
IN USE	Stage 2 passed the current call. A call is actively in-flight. Can return to ACTIVE (session reuse) or advance to EXPIRED / ABORTED.
EXPIRED	Token TTL elapsed. Terminal. No further calls. Agent must request a new token from the Authority for the next session.
REVOKED	Authority pushed a revocation signal via WatchRevocations. Terminal. Revocation cache updated immediately. In-flight calls blocked at Stage 1.
ABORTED	Authority pushed an abort signal via WatchAborts. Terminal. In-flight call terminated immediately. ABORT audit event emitted.

> Transition ownership: Stage 1 drives ISSUED → ACTIVE. Stage 2 drives ACTIVE → IN USE. TTL drives EXPIRED. The Authority (FA push) drives REVOKED and ABORTED.

8.1 Token Lifecycle vs Session Lifecycle

These are not the same thing and confusing them leads to incorrect agent behaviour.

Token lifecycle	Session lifecycle
Managed by Firma (issued, validated, expired, revoked)	Managed by the agent runtime and application
TTL is set by Mini Authority (or FA) at IssueCapability time	Session duration is determined by the agent's task
On EXPIRED: agent must call IssueCapability again to get a new token	On session end: agent disposes the token; FA closes the session context
Multi-use: one token covers N calls within its TTL	One session maps to one token at a time
If token expires while session is live: Stage 1 denies the next call. Agent must re-authenticate. Sidecar does not auto-renew.	If session ends before token expires: token should be explicitly invalidated by calling FA, not left to expire silently

In V1, the Sidecar does not auto-renew tokens. When Stage 1 returns a DENY with reason TOKEN_EXPIRED, the agent's session management code must call IssueCapability again and resume with the new token. Design agent session managers to handle this gracefully.

9. Cedar Policies · Schema · Examples

Policy content and documentation that make the OSS package usable from day one. Not a runtime component. Without working Cedar files, no one can write policies. Without a getting-started guide, adoption fails at the first README read.

9.1 Cedar schema base

Defines entity types (Agent, Session, Resource, Connector, MemoryNamespace), action types (APICall, ToolUse, MemoryRead, MemoryWrite, etc.), and context attributes that the CEE assembles and makes available at eval time (budget_remaining, risk_score, session_duration_s, action_count, etc.). This schema is the stable interface between the Firma runtime and operator-written policies.

9.2 Example policies

`allow specific API call`	Permit an agent to call a specific endpoint with a specific HTTP method. Starting point for any integration.
`deny all (default)`	Default-deny base policy. Every deployment must include this. No other policy → deny.
`memory namespace isolation`	Prevent cross-namespace memory access. Essential for multi-agent / multi-tenant deployments.
`budget limit policy`	Deny calls when cumulative token or API-call budget is exceeded. Uses pre-computed context attribute.
`risk threshold policy`	Deny calls whose static risk attribute exceeds a configured threshold. V1: threshold check only, no dynamic scoring.

9.3 Documentation and contracts

- Proto / API contracts — canonical .proto definitions for SidecarAuthority gRPC service and ExecutionEvent audit message. Stable boundary between Sidecar and any external system.
- Integration README — step-by-step: connect an agent, write first Cedar policy, read first audit event.
- Getting started guide — zero to running end-to-end demo with firma dev up.

10. Local Dev Mode · firma dev up

Orchestration command that starts Mini Authority (:50051), Sidecar (:8080), and Example Agent (:8888) in a pre-wired configuration. End-to-end in under 2 minutes, zero cloud dependency.

Features: live Cedar policy hot-reload (edit .cedar while running, bundle pushed to Sidecar automatically), audit log to stdout, and all three example scenarios (ALLOW / DENY / ABORT) runnable immediately.

> Local Dev Mode is the recommended starting point. It makes the full execution pattern observable — pre-flight, Stage 1, Stage 2 / CEE, Connector — before any production integration work begins.

11. Performance Targets (V1)

These targets define the expected operating envelope of the Sidecar in a standard single-instance deployment. They serve two purposes: guiding initial implementation decisions, and establishing regression baselines for benchmarking and CI.

Metric	Target	Notes	Measurement
Stage 1 latency	**< 1 ms p95**	Token parse + sig verify + bloom revoc check. No network I/O. Dominated by ECDSA verify (~200 µs on commodity hardware).	Benchmark: isolated Stage 1 microbench
Stage 2 latency (CEE)	**< 200 µs p95**	Context assembly + Cedar eval. Cedar is deterministic; eval time scales with policy complexity. Simple allow/deny rules are < 50 µs.	Benchmark: Cedar eval with reference policy set
End-to-end overhead	**< 3 ms p95**	Total Sidecar-added latency per call: interceptor + Stage 1 + Stage 2 + credential injection + audit emit (async). Excludes connector and external system latency.	Measured: agent → Sidecar → connector entry
Memory footprint	**< 100 MB RSS**	Includes policy bundle cache, revocation LRU cache, in-flight request state, and audit WAL buffer. Scales with policy bundle size and concurrent session count.	Profiled: steady-state under load
Throughput (single instance)	**5k – 20k req/s**	Lower bound: conservative Cedar policies, small bundles. Upper bound: simple allow/deny, minimal context. Scales horizontally; each agent process gets its own Sidecar instance.	Load test: wrk2 / ghz against mock connector

Policy bundle hot-reload	**< 500 ms**	Time from WatchPolicyBundle push received to CEE using the new bundle. Includes deserialisation and atomic swap. In-flight calls complete against the previous bundle.	Measured: bundle push → first eval with new bundle
Revocation propagation	**< 1 s p99**	Time from Authority pushing a revocation event to Stage 1 rejecting the revoked token. Bounded by gRPC stream delivery + bloom filter update.	Measured: revoc push → first Stage 1 DENY

> These targets apply to a standard single-core deployment. They are not hard SLAs for V1. The implementation team should treat them as design constraints and regression gates. If Stage 1 exceeds 1 ms p95 in CI benchmarks, investigate before merging.

12. Failure Modes

These are the failure scenarios every team deploying the Sidecar must handle. The decision column states the Sidecar's default behaviour. Operators can override where noted.

Failure scenario	Decision	Sidecar behaviour	Notes
Mini Authority down at session start	**Fail closed**	IssueCapability RPC fails. Sidecar returns DENY to agent with reason AUTHORITY_UNAVAILABLE. No token issued, no calls proceed. Agent must retry with backoff.	Expected: FA replaces Mini Auth in prod. Same behaviour.
WatchPolicyBundle disconnected — TTL not yet expired	**Continue (degraded)**	Sidecar continues serving requests against the cached bundle. Logs a warning every N seconds. No new calls blocked yet.	Default TTL: 30 s. Configurable.
WatchPolicyBundle disconnected — TTL expired	**Fail closed**	Sidecar enters fail-closed mode. All new requests denied with POLICY_BUNDLE_STALE. Existing in-flight calls complete. Resumes on stream reconnect.	TTL reset on reconnect + first bundle push.
WatchRevocations delayed / disconnected	**Continue (degraded)**	Sidecar cannot receive new revocations. Existing cache still valid. Logs warning. Considered acceptable degradation for TTL-bounded window.	Operators can choose fail-closed via config flag.
Audit sink unavailable (gRPC)	**WAL buffer**	Events written to local WAL. Sidecar continues accepting calls. WAL replayed when sink reconnects. WAL size capped; if	File / stdout sinks never fail silently.

		cap exceeded, oldest events are dropped and a counter is emitted.	
Connector timeout	**Abort in-flight**	Connector returns timeout error. Sidecar emits ABORT audit event, returns CONNECTOR_TIMEOUT to agent. Token state: IN USE → ACTIVE (call is treated as not completed).	Connector timeout configurable per-connector.
Vault / credential injector unavailable	**Fail closed**	Credential injection fails. Call blocked with CREDENTIAL_INJECTION_FAILED. Agent receives DENY. No call dispatched to external system.	Basic cred injection from config never fails this way.

13. V1 Scope Boundary — What is NOT in OSS

The following capabilities are intentionally excluded from V1. This section exists to prevent teams from building expectations that will be disappointed, and to avoid premature complexity in the OSS codebase.

`Trust graph`	No dynamic trust relationship modelling between agents, sessions, or principals. Not in Mini Authority, not in the Sidecar.
`Dynamic risk engine`	No real-time risk scoring. V1 risk is a static attribute threshold-check. A live risk engine is a Firma Authority (production) capability.
`Multi-tenant control plane`	Mini Authority is single-tenant, file-based. No per-org policy namespacing, no tenant isolation in the authority layer.
`Compliance-grade audit backend`	The audit emitter produces signed events. It does not provide a tamper-evident store, replay index, or compliance reporting. FirmaChain covers this in production.
`Enterprise memory governance`	Memory namespace isolation is provided via Cedar policy. Enterprise-grade memory provenance, lineage tracking, and cross-session memory governance are not in scope.
`Cedar policy compiler / UI`	The OSS release ships .cedar source files. The F-Control Plane (policy authoring UI, compiler pipeline, bundle distribution) is a proprietary capability.
`Escalation engine`	No human-in-the-loop escalation path in V1. Agents that require escalation for out-of-policy requests must implement this at the application layer.

`provenance chain (V1)`	The Execution Envelope schema includes a provenance field, but the V1 runtime does not populate or verify a hash chain. Treat as a reserved, nullable field.

> For production deployments: replace Mini Authority with Firma Authority (config-only swap), route audit events to FirmaChain, and attach the F-Control Plane for policy authoring and bundle distribution. The Sidecar binary is identical in both environments.